

DEPLOYMENT GUIDE

TrustLedger Production Deployment

Step-by-step guide to deploying the TrustLedger AI Code Governance Platform to Vercel with Supabase, Upstash Redis, Stripe, and Sentry.

VERSION

1.0.0

DATE

8 June 2026

PLATFORM

Vercel + Supabase

RUNTIME

Node.js 20 · Next.js 13.5

Contents

This guide covers every step from zero to a live production deployment.

1	Prerequisites & Architecture Overview	3
2	Supabase — Database, Auth & Realtime	4
3	Upstash Redis — Rate Limiting	6
4	GitHub App — Webhook Integration	7
5	Vercel — Deploy the Application	8
6	Environment Variables Reference	10
7	Vercel Cron Jobs	12
8	Optional Services (Stripe, SendGrid...)	13
9	Post-Deploy Verification	15
10	Docker / Self-Hosted Alternative	16
11	Troubleshooting	17
12	Go-Live Checklist	18

1 Prerequisites & Architecture Overview

Review what you need before starting and understand how the pieces fit together.

1.1 What you need

- A GitHub account with access to the TrustLedger repository
- A Vercel account (free tier works; Pro required for cron jobs on custom domains)
- A Supabase account (free tier works for staging; Pro recommended for production)
- An Upstash account (free tier Redis is sufficient)
- Node.js 20+ installed locally (for running migrations)
- npm 9+ installed locally

1.2 Architecture at a glance

TrustLedger is a Next.js 13.5 application that uses the App Router with both server components and client components. All API routes live under /api and run as Vercel Serverless Functions. Background work runs via five Vercel Cron Jobs.

Browser / GitHub Webhook

¼

Vercel (Next.js 13.5) • Vercel Crons (5 schedules)

¼

¼

Supabase (Postgres, Auth, RT) Upstash Redis (Rate limiting, idempotency)

Stripe (billing webhooks)
SendGrid (alert emails)
Sentry (error tracking)
PostHog (product analytics)

1.3 Clone the repository

```
git clone https://github.com/your-org/trustledger.git
cd trustledger/dashboard
```

```
npm install
```

Important

All commands in this guide assume you are inside the dashboard/ directory unless stated otherwise.

2 Supabase — Database, Auth & Realtime

Supabase provides the PostgreSQL database, authentication, row-level security, and real-time subscriptions used by the live dashboard.

2.1 Create a Supabase project

- Go to supabase.com and sign in
- Click New Project, choose your organisation, set a name (e.g. trustledger-prod) and a strong database password
- Select the AWS region closest to your users (the app defaults to us-east-1 / iad1)
- Wait ~2 minutes for provisioning to complete

2.2 Collect your API credentials

Navigate to Project Settings ' API and copy the following values — you will need them later:

Value	Where to find it
SUPABASE_URL	Settings ' API ' Project URL
SUPABASE_ANON_KEY	Settings ' API ' Project API Keys ' anon public
SUPABASE_SERVICE_ROLE_KEY	Settings ' API ' Project API Keys ' service_role (keep secret)

2.3 Run database migrations

The migration script creates all tables, indexes, RLS policies, and seed data. Set the environment variable first, then run:

```
# Set the connection string (find it in Settings ' Database ' Connection string)
export DATABASE_URL="postgresql://postgres:[password]@db.[ref].supabase.co:5432/postgres"

npm run migrate
# Verify all migrations applied:
npm run migrate:status
```

Migration dry-run

Run `npm run migrate:dry` first to preview SQL without applying it. Inspect the output before running against production.

2.4 Configure authentication

- Go to Authentication ' Providers in the Supabase dashboard
- Enable GitHub OAuth: create a GitHub OAuth App at github.com/settings/developers, set the callback URL to `https://your-domain.com/auth/callback` and paste the Client ID and Secret into Supabase
- Optionally enable Email/Password auth for non-GitHub users
- Under Authentication ' URL Configuration, set Site URL to `https://your-domain.com` and add `https://your-domain.com/auth/callback` to Redirect URLs

2.5 Enable Realtime

The live dashboard uses Supabase Realtime to push updates without polling. Enable it for each of these tables in Database ' Replication:

- scans
- attestations
- violations
- alerts
- incidents

3 Upstash Redis — Rate Limiting

Upstash provides a serverless Redis instance used for API rate limiting and webhook idempotency. Without it the app falls back to in-memory rate limiting, which is not safe across multiple Vercel function instances.

3.1 Create a Redis database

- Go to console.upstash.com and sign in
- Click Create Database, choose a name (e.g. `trustledger-ratelimit`) and select the AWS region that matches your Vercel deployment (`us-east-1` recommended)
- Leave the type as Regional (not Global) unless you have multi-region Vercel deployments
- Click Create

3.2 Collect credentials

From the database detail page, copy:

- `UPSTASH_REDIS_REST_URL` — the HTTPS REST URL
- `UPSTASH_REDIS_REST_TOKEN` — the REST token

3.3 Rate limit configuration

The built-in rate limits are defined in `src/lib/rateLimit.ts`. Default values:

Endpoint type	Limit	Window
Scan submissions	60 requests	60 seconds
API key auth	300 requests	60 seconds
Webhook ingest	100 requests	60 seconds

Free tier is sufficient

Upstash free tier supports 10,000 commands/day and 256 MB storage. This comfortably handles hundreds of scans per day. Upgrade when you exceed ~1,000 API calls per hour.

4

GitHub App — Webhook Integration

The GitHub App receives pull request webhook events, triggers scans, and posts Check Run results back to GitHub. This is required for the core PR-gate workflow.

4.1 Create the GitHub App

- Go to github.com/settings/apps (personal) or github.com/organizations/YOUR_ORG/settings/apps (org)
- Click New GitHub App
- Set Homepage URL to <https://your-domain.com>
- Set Webhook URL to <https://your-domain.com/api/github-app>
- Generate a random Webhook secret (e.g. `openssl rand -hex 32`) and save it

4.2 Set permissions

Under Permissions & Events, grant the following:

Permission	Level
Repository: Pull requests	Read & write
Repository: Checks	Read & write
Repository: Contents	Read only
Repository: Statuses	Read & write

Under Subscribe to events, check: Pull request, Push

4.3 Generate a private key

- After creating the app, scroll to Private keys and click Generate a private key
- Download the .pem file
- Convert it for use as an env var: `cat private-key.pem | base64 | tr -d "\n"`
- The base64 output becomes `GITHUB_APP_PRIVATE_KEY`

4.4 Install the app

- Click Install App in the left sidebar and install it on the repositories you want TrustLedger to scan
- Copy the App ID from the General settings page — this becomes `GITHUB_APP_ID`

GitLab / Bitbucket

TrustLedger also supports GitLab MR scanning (via /api/gitlab-webhook) and Bitbucket PR scanning (/api/bitbucket-webhook). Configure those webhooks similarly using the GITLAB_WEBHOOK_SECRET and BITBUCKET_WEBHOOK_SECRET env vars.

5 Vercel — Deploy the Application

The project ships with a `vercel.json` that configures the build, security headers, deployment region, and cron jobs. Deployment is a single command.

5.1 Install the Vercel CLI

```
npm install -g vercel
vercel login
```

5.2 Link your project

```
# From the dashboard/ directory:
vercel link
# Answer the prompts: select your team/scope and project name
```

5.3 Add environment variables

Add each variable via the Vercel CLI or the dashboard (Project ' Settings ' Environment Variables). Use the CLI for batch entry:

```
# Repeat for each variable — CLI prompts for the value:
vercel env add NEXT_PUBLIC_SUPABASE_URL production
vercel env add NEXT_PUBLIC_SUPABASE_ANON_KEY production
vercel env add SUPABASE_SERVICE_ROLE_KEY production
vercel env add NEXT_PUBLIC_APP_URL production
vercel env add NEXT_PUBLIC_SKIP_AUTH production
# ... (see full reference on page 10)
```

Critical: disable demo mode

Set `NEXT_PUBLIC_SKIP_AUTH=false` in production. The app will refuse to start if this is true in a production `NODE_ENV` environment.

5.4 Deploy

```
# Preview deployment (staging):  
vercel  
  
# Production deployment:  
vercel --prod
```

5.5 Connect a custom domain

- In Vercel dashboard: Project ' Settings ' Domains ' Add
- Enter your domain (e.g. app.trustledger.dev)
- Add the CNAME or A record shown to your DNS provider
- Wait for SSL provisioning (~2 minutes for Let's Encrypt)
- Update NEXT_PUBLIC_APP_URL to match the new domain and redeploy

5.6 Configure the Supabase Auth callback

After setting your custom domain, go back to Supabase ' Authentication ' URL Configuration and update Site URL and Redirect URLs to your production domain. GitHub OAuth also needs the callback URL updated in the GitHub OAuth App settings.

5.7 Verify the build

```
vercel logs --follow  
# Look for: Ready in Xs  
# Then visit /healthz to confirm DB connectivity
```

6

Environment Variables Reference

Complete list of all environment variables. Required variables must be set before deployment. Optional variables enable additional features but the app runs without them.

Core (required)

Variable	Status	Notes
NEXT_PUBLIC_SUPABASE_URL	Required	Your Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Required	Supabase anon/public key
SUPABASE_SERVICE_ROLE_KEY	Required	Supabase service role key — server only, never expose to browser
NEXT_PUBLIC_APP_URL	Required	Full URL of your deployment e.g. https://app.example.com
NEXT_PUBLIC_SKIP_AUTH	Required	Must be false in production. true only for local demo mode.
NEXT_PUBLIC_ORG	Required	Your organisation slug e.g. acmecorp — used in mock data and API scoping
UPSTASH_REDIS_REST_URL	Required	Upstash Redis REST endpoint URL
UPSTASH_REDIS_REST_TOKEN	Required	Upstash Redis REST token
CRON_SECRET	Required	Random 32-char string. Cron requests must pass Bearer {CRON_SECRET}

GitHub integration

Variable	Status	Notes
GITHUB_WEBHOOK_SECRET	Required	HMAC-SHA256 secret set when creating the GitHub App webhook
GITHUB_APP_ID	Required	Numeric App ID from the GitHub App general settings page
GITHUB_APP_PRIVATE_KEY	Required	Base64-encoded PEM private key downloaded from GitHub App

Billing (optional)

Variable	Status	Notes
STRIPE_SECRET_KEY	Optional	Stripe secret key (sk_live_...)
STRIPE_WEBHOOK_SECRET	Optional	Stripe webhook signing secret (whsec_...)
STRIPE_PRICE_STARTER_MONTHLY	Optional	Stripe Price ID for Starter plan monthly
STRIPE_PRICE_STARTER_ANNUAL	Optional	Stripe Price ID for Starter plan annual
STRIPE_PRICE_GROWTH_MONTHLY	Optional	Stripe Price ID for Growth plan monthly
STRIPE_PRICE_GROWTH_ANNUAL	Optional	Stripe Price ID for Growth plan annual

Notifications & email (optional)

Variable	Status	Notes
SENDGRID_API_KEY	Optional	SendGrid API key for alert and digest emails
SENDGRID_FROM_EMAIL	Optional	Verified sender address e.g. alerts@trustledger.dev

Observability (optional)

Variable	Status	Notes
NEXT_PUBLIC_SENTRY_DSN	Optional	Sentry DSN for client-side error tracking
SENTRY_AUTH_TOKEN	Optional	Sentry auth token for source map upload during build
SENTRY_ORG	Optional	Your Sentry organisation slug
SENTRY_PROJECT	Optional	Your Sentry project slug
NEXT_PUBLIC_POSTHOG_KEY	Optional	PostHog project API key
NEXT_PUBLIC_POSTHOG_HOST	Optional	PostHog host, default https://app.posthog.com

Security headers note

Security headers (HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy) are set in both vercel.json and next.config.mjs and apply automatically. No additional configuration is needed.

Secret management

Never commit `.env.local` or any file containing real keys to git. Use `vercel env add` to inject secrets at deploy time. Rotate the `SUPABASE_SERVICE_ROLE_KEY` and `CRON_SECRET` before your first production deployment if they were ever shared.

7

Vercel Cron Jobs

Five cron jobs are pre-configured in `vercel.json`. They activate automatically on deployment. Each calls an API route that verifies the `CRON_SECRET` bearer token.

Path	Schedule	Purpose
<code>/api/cron/sla</code>	Every 15 min	Scans open violations for SLA deadline breaches. Fires alerts for overdue items and updates violation statuses.
<code>/api/cron/webhook-retry</code>	Every 5 min	Retries failed outbound webhooks (GitHub Check Run updates, Slack notifications) with exponential back-off.
<code>/api/cron/scheduled-scans</code>	Hourly	Runs scheduled repository scans configured via the integrations settings page.
<code>/api/cron/weekly-report</code>	Mon 8am UTC	Generates and emails the weekly compliance digest report to configured recipients.
<code>/api/cron/compliance-reminders</code>	Daily 9am UTC	Sends reminders for upcoming compliance calendar deadlines (audits, cert expiry, reviews).

How crons are authenticated

Vercel sends an `Authorization: Bearer` header containing your `CRON_SECRET` value. Each cron handler in the API verifies this before executing. If `CRON_SECRET` is not set or is set to the default `dev-cron-secret` value the go-live checker will warn you.

Monitoring crons

- Vercel dashboard 'Project' Cron Jobs shows execution history and errors
- Failed cron runs trigger Sentry errors if `NEXT_PUBLIC_SENTRY_DSN` is configured
- Crons only run on Vercel Pro or above for custom domains. They work on all plans for the `*.vercel.app` domain.

Local cron testing

To test a cron handler locally, call it directly with curl: `curl -H "Authorization: Bearer dev-cron-secret" http://localhost:3000/api/cron/sla`

8

Optional Services

These services enhance the platform but are not required to get started. Configure them when you are ready to move beyond basic functionality.

8.1 Stripe — Billing

- Create a Stripe account and go to the Developers ' API Keys page
- Copy your live secret key (sk_live_...) as STRIPE_SECRET_KEY
- Create three products (Starter, Growth, Enterprise) in Stripe ' Products, each with monthly and annual prices
- Copy each Price ID (price_...) into the STRIPE_PRICE_* env vars
- Set up a webhook in Stripe ' Developers ' Webhooks pointing to https://your-domain.com/api/stripe
- Subscribe to: checkout.session.completed, customer.subscription.updated, customer.subscription.deleted, invoice.payment_failed
- Copy the webhook signing secret (whsec_...) as STRIPE_WEBHOOK_SECRET

8.2 SendGrid — Alert emails

- Create a SendGrid account and go to Settings ' API Keys ' Create API Key (Full Access)
- Set SENDGRID_API_KEY to the generated key
- Verify a sender email address in Settings ' Sender Authentication and set it as SENDGRID_FROM_EMAIL
- Alert emails are sent for P1 incidents, SLA breaches, and weekly compliance digests

8.3 Sentry — Error tracking

- Create a Sentry project (platform: Next.js)
- Copy the DSN from Project ' Settings ' Client Keys as NEXT_PUBLIC_SENTRY_DSN
- For source map upload: create an Auth Token in User Settings ' Auth Tokens with project:releases and org:read scopes. Set SENTRY_AUTH_TOKEN, SENTRY_ORG, SENTRY_PROJECT
- The app uses @sentry/nextjs and captures both client errors (via the global error boundary) and server errors (in API routes)

8.4 PostHog — Product analytics

- Create a PostHog Cloud project at app.posthog.com
- Copy the Project API Key as NEXT_PUBLIC_POSTHOG_KEY
- Leave NEXT_PUBLIC_POSTHOG_HOST unset to use the default EU cloud endpoint, or set it for self-hosted PostHog
-

Page views, feature interactions, and scan events are tracked automatically

Ship without these

The core scan, attest, audit, and compliance features work without Stripe, SendGrid, Sentry, or PostHog. Add them progressively as you onboard real users.

9

Post-Deploy Verification

After the first successful deployment, run these checks to confirm everything is wired up correctly.

9.1 Health endpoint

```
curl https://your-domain.com/healthz
# Expected response:
{ "status": "ok", "db": "connected", "latency_ms": 12 }
```

If db is not "connected", check that SUPABASE_SERVICE_ROLE_KEY is set correctly and that the Supabase project is not paused.

9.2 Run the go-live checklist

Visit <https://your-domain.com/admin/go-live> while logged in as an admin. The page runs automated checks across five categories:

- Infrastructure — env vars set, skip-auth disabled
- API Health — /healthz responds, database connected
- Security — HTTPS, cron secret, webhook secret
- Features — Stripe, SendGrid, Redis configured
- Observability — Sentry, PostHog active

All required items must show PASS before taking production traffic. WARN items are optional services — resolve them before onboarding paying customers.

9.3 Seed demo data (optional)

Visit /seed to apply mock scan data for a demo walkthrough. This writes to localStorage and does not affect the database. Clear it by visiting /seed again and clicking Clear.

9.4 Submit a test scan

```
curl -X POST https://your-domain.com/api/scans \
-H "Authorization: Bearer YOUR_API_KEY" \
-H "Content-Type: application/json" \
-d '{"repo":"org/test","commit_sha":"abc123","files":[]}'
```

```
# Should return: { scan_id: "...", overall_risk: "LOW", ... }
```

API keys

Generate API keys in Settings ' Integrations ' API Keys. Each key is scoped to an organisation and rate-limited. The key format is `tl_live_...` for production.

10 Docker / Self-Hosted Alternative

The Next.js config sets output: "standalone" which produces a minimal, self-contained Node.js server. Use this path for AWS ECS, GCP Cloud Run, Kubernetes, or any VPS.

10.1 Build the standalone output

```
npm run build
# Output: .next/standalone/ — a minimal server with no node_modules/
```

10.2 Dockerfile

```
FROM node:20-alpine AS base
WORKDIR /app

# Copy standalone build output
COPY .next/standalone ./
COPY .next/static ./next/static
COPY public ./public

ENV NODE_ENV=production
ENV PORT=3000
EXPOSE 3000

CMD ["node", "server.js"]
```

```
docker build -t trustledger:latest .
docker run -p 3000:3000 \
-e NEXT_PUBLIC_SUPABASE_URL=... \
-e SUPABASE_SERVICE_ROLE_KEY=... \
# (pass all env vars via -e or --env-file .env.production)
trustledger:latest
```

10.3 Cron jobs on self-hosted

Vercel crons do not run in a Docker deployment. Use one of these alternatives:

- Kubernetes CronJob resource — one job per endpoint, each calling curl with the CRON_SECRET header
- AWS EventBridge Scheduler — trigger an HTTP call to each cron path on the same schedule as vercel.json
- GCP Cloud Scheduler — same approach, one job per path
- A sidecar cron container running crontab that calls the API endpoints

Reverse proxy

Place an nginx or Caddy reverse proxy in front of the Docker container to handle SSL termination and the HSTS headers. The Next.js server itself serves on plain HTTP and expects the proxy to terminate TLS.

11 Troubleshooting

Common issues encountered during deployment and how to resolve them.

Build fails: "Cannot find module"

Usually caused by a missing or mismatched Node.js version. The project requires Node 20+. Check with `node --version`. On Vercel set Node.js Version to 20.x in Project ' Settings ' General.

500 errors after deploy

Check Vercel Functions logs (Project ' Functions ' select function ' Logs). The most common cause is a missing environment variable. Look for "Cannot read properties of undefined" or "missing required env" in the logs.

Supabase "JWT expired" or auth errors

Ensure `NEXT_PUBLIC_SUPABASE_ANON_KEY` and `SUPABASE_SERVICE_ROLE_KEY` are set to the correct project. If you regenerated keys in Supabase, redeploy Vercel to pick up the new values.

Realtime not updating

- Check that Realtime is enabled for the relevant tables in Supabase ' Database ' Replication
- Ensure the Supabase project is on a plan that supports Realtime (free tier is limited to 2 concurrent connections)
- Check browser console for WebSocket connection errors

GitHub webhooks returning 401

The HMAC-SHA256 signature validation failed. Verify that `GITHUB_WEBHOOK_SECRET` exactly matches the secret set in the GitHub App webhook settings. No trailing spaces or newlines.

Cron jobs not running

- Crons require Vercel Pro for custom domains — they run for free on `*.vercel.app`
- Verify `CRON_SECRET` is set and matches what the cron handlers expect
- Check Vercel dashboard ' Project ' Cron Jobs for execution history and error details

Rate limit errors (429)

If `UPSTASH_REDIS_REST_URL` is not set the app uses in-memory rate limiting. Under load this can cause false 429s because each serverless function instance has its own counter. Set the Upstash env vars to use shared Redis counters.

Getting help

Check the go-live checklist at `/admin/go-live` first — it surfaces most configuration problems automatically. For persistent issues check the Vercel function logs and Supabase logs (Database ' Logs) side by side.

12

Go-Live Checklist

Print this page and work through it before directing production traffic to your deployment. Every item must be checked before going live.

Infrastructure

- ☐ Supabase project created and database migrations applied (npm run migrate:status shows all green)
- ☐ NEXT_PUBLIC_SUPABASE_URL and NEXT_PUBLIC_SUPABASE_ANON_KEY set in Vercel
- ☐ SUPABASE_SERVICE_ROLE_KEY set in Vercel (server-only, not prefixed NEXT_PUBLIC_)
- ☐ NEXT_PUBLIC_APP_URL set to the production HTTPS domain
- ☐ NEXT_PUBLIC_SKIP_AUTH set to false
- ☐ Custom domain added to Vercel and SSL certificate issued
- ☐ Supabase Auth callback URL updated to production domain

Rate limiting & security

- ☐ UPSTASH_REDIS_REST_URL and UPSTASH_REDIS_REST_TOKEN set
- ☐ CRON_SECRET set to a unique random string (not the default dev-cron-secret)
- ☐ GITHUB_WEBHOOK_SECRET set and matches the GitHub App webhook configuration
- ☐ GITHUB_APP_ID and GITHUB_APP_PRIVATE_KEY set

Verification

- ☐ /healthz returns { status: "ok", db: "connected" }
- ☐ /admin/go-live shows all required checks as PASS
- ☐ A test scan submitted via the API returns a valid scan_id
- ☐ GitHub webhook delivery test (GitHub App 'Advanced' Recent Deliveries) shows 200
- ☐ Realtime: open two browser tabs and confirm scan status updates propagate live

Optional (before onboarding customers)

- ☐ Stripe billing configured and a test checkout completed
- ☐ SendGrid sender verified and a test alert email received

- ☐ Sentry DSN set and a test error captured
 - ☐ PostHog key set and page views appearing in the dashboard
 - ☐ Cron jobs visible in Vercel dashboard with at least one successful execution
 - ☐ Seed data cleared from any test accounts (/seed ' Clear)
-

Once all boxes above are checked, TrustLedger is production-ready.
Visit /dashboard and onboard your first repository.